

Deterministic Genesis: Coordination-Free Structural Initialization for Operation-Based CRDTs

Merkle, B.¹ Rodionov, V.²

Draft — preprint, April 2026

Abstract

Every CRDT-backed application must establish structural state — containers, schemas, defaults — before user operations can proceed. In decentralized settings, this initialization is a coordination problem: two peers independently creating the same structure produce distinct CRDT operations that conflict rather than converge. We present deterministic genesis, a technique where structural entities are created via content-addressed replica identifiers in throwaway documents, guaranteeing that independent peers produce byte-identical operations under a factory purity assumption. Sync becomes a structural no-op. The technique composes with semantic identifier partitioning^[1] and the entity-oriented document model^[2] to provide undo invisibility and content-addressed naming for structural entities. Deployed in a production CRDT framework (Plexus/Yjs).

1 The Initialization Problem

CRDT-backed collaborative applications require structural state — containers, schemas, defaults — before user editing begins. A text editor needs a root paragraph. A design tool needs layer containers. A component system needs type maps and schema containers. We use the term *genesis* to refer to the deterministic creation of these structural entities via content-addressed replica identifiers, and *genesis entities* for the entities so produced. Neither *genesis* (in this specific CRDT sense) nor the companion term *liminal* (in the sibling paper on deferred-persistence lifecycles) is established in the CRDT literature; both are coinages introduced by this paper stack.

In centralized architectures, a server creates these structural entities authoritatively. In decentralized, offline-first architectures, there is no authority — and the problem becomes acute.

1.1 What Goes Wrong

Consider two peers, Alice and Bob, both opening the same document for the first time while offline.

Duplication. Both peers create a root paragraph containing empty text. Each creation is a distinct CRDT operation (different replica identifiers, different Lamport clocks). On sync, the document contains two root paragraphs. The Yjs community documents this

directly: “two people enter a document in an offline state... both their documents are initialized with the initial document value. When one goes online, all good. When the other goes online, this initial document is duplicated”^[3].

A related failure is silent data loss through structural divergence. If both peers create a `config` map under the key `"settings"`, with Alice writing `config.theme = "dark"` and Bob writing `config.language = "fr"`, the CRDT resolves the key conflict on sync by selecting one value based on replica identifier ordering — discarding the other map entirely. Alice’s `theme` or Bob’s `language` is permanently lost, not because the values conflict but because the *containers* are distinct CRDT objects despite being semantically identical; the writes targeted different physical maps. No error, no conflict marker, no notification.

The same pattern shows up in schema evolution: adding a field to a document type requires creating a new container on every existing document, and if two peers independently apply the same migration they produce distinct containers — the duplication problem at the schema level. A Yjs community developer describes this as “the biggest glaring flaw with Yjs... I’d wager supporting local-first back compat makes dev take 50% longer”^[4].

Undo corruption. Structural entities created during initialization are captured by the undo manager alongside user operations. A user pressing Ctrl+Z may inadvertently destroy a schema container or root element — producing an invalid document state. Every major framework addresses this with ad hoc filtering: Yjs uses `trackedOrigins`,

¹Independent researcher, Here.build; x.com/merkle_bonsai; ImmuneFi top-40 whitehat (ethical hacker); email: merklebonsai@pm.me

²Independent researcher, Here.build; email: v@here.build

Loro uses `excludeOriginPrefixes`, ProseMirror uses `addToHistory: false`. None provides a structural guarantee that initialization entities are permanent. Dolan^[5] proves that algebraic undo for general CRDTs is impossible beyond counters, making filtering the only option — but every filter is an allowlist maintained by convention, not by the type system.

1.2 Existing Workarounds

Pre-encoded binary templates. Both Yjs^[3] and Automerge^[6] arrive at the same recommendation: create the initial state once, serialize to a byte array, and hard-code that array into the application. Every peer applies the identical binary via `Y.applyUpdate()` or `Automerge.load()`. Because the binary is byte-identical across peers, the CRDT deduplicates on merge. This works for static initial state known at build time, but fails for per-entity initialization (a new component needs its own containers), schema evolution (new fields added after deployment), and any state that depends on the document’s existing content.

Server-authoritative initialization is the next step up the coordination ladder: Liveblocks reads `initialStorage` once when a room is first entered and the server ensures single initialization^[7], and Figma avoids the problem entirely by using a central server as the authority^[8]. Both solutions sacrifice offline-first capability, and Liveblocks handles single-initialization but not per-entity initialization — the generalization genesis provides. At the other extreme, some Yjs community members discovered a *clientID=0 hack*: temporarily setting `doc.clientID = 0` on all peers before initialization writes makes the operations appear to come from the same source. Kevin Jahns, creator of Yjs, explicitly warns against this: “THIS CODE IS DISCOURAGED AND WILL LIKELY BREAK YOUR Yjs DOCUMENTS...if you generate complex Y.XML documents like this, the CRDT items will have different types on different peers...the document might get permanently corrupted without a way to recover”^[3]. A third option, suggested by Jahns himself, is to layer “consistent hashing by GUID or Redis-redlock”^[3] over initialization — which requires centralized infrastructure, defeating the purpose of decentralized CRDTs.

Cambria’s phantom change. Ink & Switch’s Cambria system^[9] generates a deterministic synthetic change with actorId `'0000000000'` to establish default values — the closest prior art to our technique. The operational difference matters: Cambria’s phantom change is a **local reconstruction artifact** — each peer

synthesizes it in-memory to read old data in the new schema, and the phantom is never persisted or transmitted. Genesis produces a **shared persistent operation** that lives in the document, syncs to peers, and is addressable via the self-resolving identity encoding (§5 of^[2]). The two approaches are structurally related but operationally distinct: Cambria uses phantom-actor identity to read-bridge schema versions; genesis uses deterministic-actor identity to write-converge structural creation. Cambria was discontinued after Automerge’s internal optimizations broke the integration; its core insight — that deterministic actor identity is admissible — survives in the genesis construction.

1.3 The Underlying Conflation

CRDT operations conflate two distinct concerns:

1. **Structural initialization** — establishing containers, schemas, and genesis entities that must exist before user operations.
2. **User content creation** — producing entities that represent a specific peer’s intent.

CRDTs handle user content correctly: different peers creating different content *should* produce different entities. But structural initialization has the opposite requirement: different peers creating the *same* structure should produce the *same* entity.

Making initialization operations convergent therefore requires a mechanism that produces physically identical CRDT items regardless of which peer runs the initialization. Post-hoc deduplication is insufficient — the items need to arrive at every peer already equal at the CRDT-identifier level.

2 Technique

Genesis operates at two layers of the stack, and this paper covers both. **Structural genesis** (§2.1–§2.2, applied in §5.1) uses only partitioning^[1] and produces the type sub-maps that anchor the entity-oriented document model of^[2]; it is a *prerequisite* for^[2]’s flat-shell architecture. **Virtual-children genesis** (§2.3, applied in §5.2) consumes^[2]’s UUID encoding (§5 of^[2]) for its `parentUuid` input and^[2]’s schema concept for its field/key inputs; it is an *application* of^[2]. Both share the content-hashing + throwaway-document mechanism, which is why this single paper covers both, but the dependency direction differs: structural genesis enables^[2], while virtual-children genesis consumes^[2].

Unlike the companion architecture^[2] and liminality^[10] papers, which describe their techniques abstractly and push Yjs-specific API usage to appendices, this paper presents the construction directly in its Yjs realization in the main body. The construction is thin — a throwaway `Y.Doc`, a deterministic `clientId`, and `applyUpdate` — so abstracting below that produces a requirements list (C7) but no additional insight. The technique generalizes to any op-based CRDT with mutable replica ID, fresh-doc state construction, and `applyUpdate`-equivalent remote-origin injection; C7 is the portability list.

2.1 Content-Addressed Replica Identifiers

Deterministic genesis uses the semantic identifier partitioning scheme^[1] to place initialization operations in a dedicated namespace — the genesis range $[3 \cdot 2^{51}, 2^{53})$. This range has the highest priority in CRDT conflict resolution, ensuring genesis items are not displaced by concurrent user operations in sequence ordering^[1].

Within this range, each structural entity receives a replica identifier derived from a content-addressed hash of its structural position:

```
genesisClientId(type, path) = hash(type, path) mod 251
    + GENESIS_BASE
```

The hash combines two 32-bit Murmur3 hashes (seeds: 0x47454e, 0x534953) into a 51-bit value: 19 bits from the first hash concatenated with 32 bits from the second. `GENESIS_BASE` = $3 \cdot 2^{51}$. Murmur3 is non-cryptographic — chosen for speed, not collision resistance (see C2, C3 for the implications).

The inputs — type name and path within the document tree — are deterministic properties of the structural position. Two independent peers computing the same genesis identifier for the same position get the same value.

2.2 Throwaway Document Isolation

Each genesis entity is produced in a throwaway CRDT document:

```
1. Create fresh Y.Doc (tmpDoc)
2. Set tmpDoc.clientId = genesisClientId(type, path)
3. Execute the factory function in tmpDoc
4. Encode: vector = Y.encodeStateAsUpdate(tmpDoc)
5. Destroy tmpDoc (let it go out of scope -- Yjs docs
   are GC'd)
6. Apply: Y.applyUpdate(realDoc, vector)
```

Because `tmpDoc` was freshly constructed, its state vector contains only the Items the factory produced — the encoded update is just the factory’s output, not a composition with any pre-existing document state.

The throwaway document guarantees:

- **Clock = 0.** A fresh document starts its Lamport clock at 0. Every genesis item has clock 0 regardless of the real document’s state. (This is a Yjs implementation property, not a CRDT specification guarantee.)
- **No interference.** The factory runs in an empty document — it cannot read or depend on the real document’s current state.
- **Binary identity.** Same `clientId` + clock 0 + same factory output = byte-identical items. Two peers executing the same genesis produce the same binary vector, assuming factory purity (§2.4) and deterministic encoding (C7).

Step 6 applies the vector as a remote update (`Y.applyUpdate`). In Yjs, the `UndoManager` captures only local transactions^[11]. Genesis items are therefore architecturally invisible to undo/redo — not filtered, not excluded, but unreachable by the undo mechanism.

When a peer receives genesis items it has already materialized independently, Yjs’s YATA algorithm^[11] compares incoming items against existing items by `{clientId, clock}` and skips duplicates. Sync is a structural no-op.

2.3 Two-Phase Content Addressing

For *virtual children* — entities whose shape depends on a factory function and are materialized lazily on first access — deterministic genesis uses two phases:

Phase 1: Content hash. Create a fresh throwaway document with `clientId` = 0. Run the factory, connect the resulting entity to the throwaway document. Extract the *individual client vector* — only the Items created by the factory (`clientId` = 0), not the full document encoding. Hash this vector (32-bit Murmur3). This hash captures the entity’s shape with minimal input and is stable across encoding format changes.

Phase 2: Deterministic create. Compute the genesis `clientId` from (parent UUID, field name, serialized map key, shape hash from Phase 1). Create a second throwaway document with this `clientId`. Run the factory again, encode, and apply to the real document.

The two-phase approach is necessary because the genesis `clientId` must be an input to Phase 2 (it

determines the Yjs item identifiers AND the entity UUID strings stored in the encoded vector), but the `clientId` should depend on the entity’s content. Phase 1 breaks this chicken-and-egg dependency by computing a content fingerprint first.

Why not single-phase with `clientId` rewriting?

The liminality system (companion paper^[10]) uses a `withRewrittenClientId` primitive to remap Item identity (`id`, `origin`, `rightOrigin`) in the struct store. We attempted to apply this to genesis: run the factory once under `clientId = 0`, then rewrite to `genesisId`. This fails because entity UUIDs are stored as *string values* inside CRDT Items (XmlElement attributes), not as Item metadata. (In the entity-oriented document model^[2], UUID strings appear as field values and as map keys in type sub-maps; they are content, not metadata.) `ClientId` rewriting operates on the structural layer (Item identity) but cannot reach the content layer (string values). Liminality’s rewrite works because the committed delta needs Items under a new `clientId` — the UUID strings inside are irrelevant. Genesis cannot use the same shortcut because the UUID IS the identity.

The cost is two factory invocations per virtual child materialization. Document-level genesis (top-level containers) uses a single phase because the content is fixed by the schema.

Shared primitive. The `withRewrittenClientId` operation used here is the same primitive that liminality’s commit path uses; it is specified in^[10] §B.1. Genesis reuses it for individual vector extraction during the content-hash phase. The convergence was discovered during implementation: two independently-motivated features decomposed into the same encoding-level primitive.

2.4 Factory Isolation

Genesis convergence requires that factories are pure functions of their inputs. We partially enforce this with a runtime allowlist: during factory execution, only PlexusModel instances created within the factory are accessible. Accessing any external model throws an invariant violation:

```
genesisAllowlist: Set<Model> | null

assertGenesisIsolation(model):
  if genesisAllowlist is null: return // not in genesis
    context (common path)
  if model not in genesisAllowlist: throw "factory
    isolation violation"
```

The guard catches the most common impurity pattern — reading state from models outside the factory.

It does not prevent closure-captured mutable state, `Date.now()`, `Math.random()`, or other ambient non-determinism. Full purity enforcement would require static analysis or a restricted factory DSL, which we leave to future work. In practice, genesis factories are short, declarative functions (typically one `new Entity({...})` call) where impurity is easily auditable.

2.5 Convergence

Claim. Under assumptions (A1)–(A4) below, two independent peers invoking `genesis(type, path)` produce byte-identical CRDT updates.

Assumptions:

- **A1 (factory purity).** The factory is a pure function of its inputs: deterministic output, no external state access, no ambient non-determinism. §2.4 partially enforces A1.
- **A2 (encoding determinism).** `Y.encodeStateAsUpdate` produces a deterministic byte sequence for a given document state (Yjs implementation property; C7).
- **A3 (fresh-document clock).** A newly constructed `Y.Doc` starts its Lamport clock at 0 (Yjs implementation property).
- **A4 (schema agreement).** Both peers register the same factory under the same `(type, path)` input tuple.

Sketch. Under A4, both peers compute identical `genesisClientId(type, path)` (§2.1, deterministic hash). Under A3, both throwaway documents start at clock 0. Under A1, both factories produce identical content when executed in empty documents with identical `clientId`s. Under A2, the resulting encoded vectors are byte-identical.

Deduplication. On merge, Yjs’s YATA algorithm identifies Items by `{clientId, clock}`. Identical `{clientId, clock}` pairs are treated as the same Item^[11]; the content is not compared. Under (A1–A4), the pairs match by construction and the content matches by purity, so merge is a structural no-op. Under A1 violation, pairs still match but content may diverge — see §4 for the resulting failure modes.

3 Properties

P1. Convergent under purity. Two peers executing the same genesis with the same pure factory produce byte-identical CRDT items. Convergence is immediate

— no reconciliation needed. This assumes: factory purity (§2.4), deterministic Yjs encoding (C7), and Yjs item-level deduplication on $\{clientId, clock\}$ match^[11].

P2. Undo invisible. Genesis items are applied via `Y.applyUpdate` (remote origin). The `UndoManager` captures only local transactions^[11]. Genesis items are therefore architecturally unreachable by undo — not filtered by an allowlist, but invisible by construction.

P3. Positional priority. Genesis `clientIds` occupy the highest namespace range^[1]. In YATA’s sequence ordering, when two items share the same causal predecessor, higher `clientId` wins rightward placement. This ensures genesis items are not displaced by concurrent user inserts at the same position. Note: for `Y.Map` key conflicts, Yjs uses a Lamport-clock-based resolution where higher clock wins. User writes to genesis-created map entries (at `clock > 0`) correctly override the genesis defaults (at `clock 0`). Priority dominance applies to structural positioning, not to value updates.

P4. Lazily materializable. Genesis entities need not exist until accessed. A virtual map can provide an infinite logical key space where every key implicitly has a value, materialized on first access via the genesis factory. This realizes Weidner’s CRDT-Valued Map pattern^[12] with the additional guarantee of convergent *identity* (same UUID across peers) and convergent *priority* (genesis positioning), beyond the value-convergence that the pattern already provides.

P5. Content-addressed identity. Each genesis entity receives a self-resolving identifier (§5 of^[2]) with the `d` prefix, encoding the genesis `clientId` and `clock` directly. The identifier is deterministic: two peers get the same identifier for the same structural entity. Cross-references to genesis entities are stable, portable, and decodable without a lookup table.

P6. Additive schema evolution. Adding a new field to a model type is equivalent to adding a new genesis path. When a peer with the updated schema materializes the new field’s container, the genesis items propagate to all peers on sync. Peers on the old schema receive the items and store them inertly — `Y.Maps` the old schema never accesses. Peers on the new schema use them immediately. No migration script, no version negotiation.

Two caveats are load-bearing. (a) P6 assumes a **monotonic schema lattice** — old schema is strictly a subset of new schema; no existing paths change shape. Modifications to existing factories (changed defaults, renamed fields, altered entity shapes) produce different genesis `clientIds` on the same paths across versions, breaking convergence. Non-additive schema evolution

still requires coordination — Cambria’s bidirectional lenses^[9] address this complementary problem. (b) **Mixed-version read coherence** is limited: if a new-schema peer writes a cross-reference to a new genesis entity, an old-schema peer receives the reference as a valid UUID resolvable to a real shell, but its schema surfaces no field pointing at that shell. The reference is dormant — not broken, not surfaced, not garbage-collected until schema catches up. Applications relying on reachability-based liveness^[2] must treat schema rollout as a reachability concern: an entity reachable by new-schema peers may appear unreachable to old-schema peers during the rollout window. (c) The state-vector entry per genesis path (C6) accumulates monotonically over schema evolution, bounded only by the number of distinct genesis paths ever introduced.

4 Failure Modes Under Purity Violation

Genesis convergence rests on factory purity (§2.4); partial enforcement catches the common case (§2.4, C1). The residual failure modes are enumerated explicitly so that the actual worst case is on the record rather than left to reviewer hypothesis:

Divergent bytes, same clientId (single-phase structural genesis). Two peers with a subtly impure factory produce different byte sequences for the same $\{clientId, clock = 0\}$. Yjs’s YATA deduplication identifies Items by $\{clientId, clock\}$ and does *not* compare content on match, so the first-integrated copy wins and the second is dropped silently. Whichever peer synced its structural genesis first has its content canonized; the other’s diverges locally until eventually superseded. This is the strictly-worst-case failure of the construction — convergence is lost, no error is raised, and the corruption is silent. It is survivable only because (a) genesis factories are typically declarative one-liners that are easy to audit, (b) the runtime guard (§2.4) catches the most common impurity pattern (external model access), and (c) a verification relay (C5) can detect divergence by recomputing.

The two-phase variant has a different failure mode: content-hash mismatch. If the factory produces different output on its second invocation, the content hash from Phase 1 does not match the Phase 2 content, and the resulting Items live at a `clientId` that does not correspond to their content. Another peer independently materializing the same virtual child computes a different Phase 1 hash and produces Items at a different `clientId` — no collision, no convergence, two separate shells for

the same logical entity. This is the §1.1 duplication failure in a narrower form.

Factory non-termination is a separate concern: a factory that does not return hangs genesis materialization on the peer that triggers it. Because the materialization may be triggered by *receiving* a reference to a virtual child, a non-terminating factory becomes a cheap DoS vector in open systems. Cooperative deployments do not need to defend against this; open deployments should apply a factory timeout and fail materialization explicitly.

Type rename. Since genesis clientIds are hashed over (`type`, `path`), renaming a type breaks convergence on all genesis paths under that type — the renamed-type peer computes different clientIds from everyone else. Type stability is a schema-level invariant distinct from the factory-purity invariant (§2.4, C1): the factory may be pure and still produce non-convergent output if its `type` input shifts. This constraint is schema-level, not encoding-level.

In all four cases, the failure mode is **silent** from the application’s perspective unless a verification layer is present. Genesis trades loud-duplication failures for silent-divergence failures; the trade is defensible only when the enforcement mechanisms (runtime guard, verification relay, schema stability) are actually deployed. The deployment guidance is therefore: genesis requires either (a) exhaustively audited pure factories or (b) a verification relay that recomputes genesis clientIds server-side. Systems that ship neither should not use genesis.

Why the trade is nonetheless worthwhile. The §1.1 failures of the status quo (duplication, data loss, schema deadlock, undo corruption) have an *environmental* failure surface: they are triggered by network partitions, offline-first sync timing, concurrent initialization events, schema rollout. That surface is unbounded — it grows with peer count, session count, and schema evolution history. Genesis’s failure surface is *code-local*: it triggers exactly when a factory violates purity or a type is renamed. Factories are short declarative functions; purity violations are auditable by static inspection or runtime guard (§2.4). Renames are schema changes, detectable by any schema tool. A server-side relay (C5) can additionally detect genesis divergence by recomputation — an option with no analog in the status quo. The environmental, unbounded divergence surface becomes a code-local, bounded, relay-verifiable one; divergence risk is not eliminated, but it is moved into a regime where the usual software-engineering discipline

(auditing, testing, schema tooling, server-side checks) can reach it.

5 Applications

5.1 Structural Scaffolding

The original motivation. Document-level containers (type maps, field containers, schema registries) are genesis entities. Every peer, on first access, materializes the same containers with the same identities. The containers exist before user operations — not because a server created them, but because the structure is a deterministic consequence of the schema.

In the entity-oriented document model^[2], these containers serve as the scaffold roots at which flat-shell entity storage is anchored: each type sub-map lives at a genesis-created path, and the “is this a type sub-map?” check that enables structural identity protection reduces to “is this container’s parent the genesis-created types map?”

5.2 Virtual Children (CRDT-Valued Maps)

A model field declared as a virtual map provides an infinite keyspace:

```
@syncing.virtual((key: string) => new Variant({ name:
    key })))
accessor variants!: VirtualMap<string, Variant>;
```

Accessing `component.variants.get("hover")` materializes a `Variant` entity with the key “hover” — if it doesn’t already exist. The materialization is deterministic: two peers accessing the same key get the same entity with the same identifier. Subsequent writes to that entity (`variant.opacity = 0.5`) are normal CRDT operations on a shared object — they target the same physical entity and merge correctly.

This eliminates the “create before use” pattern that plagues CRDT applications. There is no “create variant” workflow, no “ensure container exists” check, no race condition. The variant exists the moment it is named.

5.3 Offline-First Default Values

The pre-encoded binary template pattern^[3,6] solves static defaults known at build time. Genesis generalizes this to dynamic, per-entity defaults that depend on structural position.

A new document opened offline by any peer materializes its full structural skeleton — type maps, root

containers, default configurations — via genesis. The skeleton is identical regardless of which peer opens the document first, second, or never. When peers connect, sync is a structural no-op.

6 Constraints

C1. Factory purity (partially enforced). Genesis convergence requires that factories are pure functions of their inputs. The runtime guard (§2.4) catches Plexus-Model field access but cannot prevent closure-captured state, ambient APIs, or non-deterministic JavaScript operations. The two-phase approach (§2.3) amplifies this: if a factory produces different output on the second invocation, the content hash from Phase 1 does not match the actual content from Phase 2, creating items with a `clientId` that does not correspond to their content.

C2. Hash collision. The genesis `clientId` is a 51-bit hash. Birthday collision probability: $P \approx n^2/2^{52}$, where n is the number of distinct genesis paths per document. For $n = 10,000$: $P \approx 10^8/4.5 \times 10^{15} \approx 2.2 \times 10^{-8}$. At fleet scale (1M documents $\times$ 10K paths), expected collisions ≈ 0.02 . A collision produces silent corruption: two entities share a `clientId`, the second materialization is a no-op (Yjs sees the items as already existing), and the second entity never appears. Murmur3 provides no resistance against targeted collisions — an adversary who controls factory inputs can craft collisions in sub-second time. **Adversarial threat model.** When hash inputs are schema-stable (type names, field names declared in source code), C2 reduces to the birthday case above. When hash inputs include user-controlled keys (virtual children keyed by user strings, §5.2), an adversary can craft colliding keys to shadow legitimate genesis entities. Defense: hash user-controlled components with SHA-256 or an equivalent cryptographic construction; Murmur3 remains acceptable only for schema-internal inputs.

Genesis paths per document	P (51-bit hash)
100	2.2×10^{-12}
1K	2.2×10^{-10}
10K	2.2×10^{-8}
1M (fleet, 1M docs $\times$ 10K paths)	~ 0.02 expected

C3. Content hash intermediate. The shape hash in two-phase content addressing (§2.3) is 32-bit Murmur3, with a birthday bound of $\sim 65K$ entities before 50% collision probability. This is the bottleneck for large schemas. Widening to 64-bit (two Murmur3 calls) would raise the bound to ~ 4 billion.

C4. Tombstone interaction. If a genesis entity is created, then deleted (tombstoned), and another peer independently materializes the same genesis entity, the incoming items match the tombstoned items' `{clientId, clock}` and remain deleted. Genesis guarantees convergent creation but does not override prior deletions. Genesis is not idempotent in the presence of tombstones.

C5. Cooperative deployment. Genesis assumes all peers use the same factory for the same structural position. A peer using a different factory produces different genesis items — breaking convergence for that path. Because genesis is deterministic, a server-side relay can enforce this by independently computing the expected genesis `clientId` for each structural position and rejecting updates with mismatched identifiers. The server does not need to trust the peer — it verifies by recomputation. **Pure P2P deployments have no such relay.** A malicious peer producing mismatched-factory items under a genesis path pollutes the document permanently — Items cannot be evicted by CRDT design, and the partition scheme's priority dominance^[1] makes the pollution conflict-winning. Genesis in adversarial P2P settings is unsafe; deployments without a verification relay require trusted-peer assumptions or out-of-band identity gating. Type renames also break convergence (the `type` input to the hash is part of the identity contract); type names must be schema-stable, which is a distinct invariant from factory purity (§2.4) and should be enforced by schema tooling.

C6. State vector inflation. Each unique genesis `clientId` adds an entry to the document's state vector (Yjs's compact summary of per-client operation counts). A document with 10K genesis paths has 10K state vector entries. This inflates sync handshake payloads proportionally.

C7. Yjs-specific assumptions. The implementation depends on: (a) item-level deduplication when applying updates with identical `{clientId, clock}` pairs^[11], (b) UndoManager ignoring remote-origin updates^[11], (c) deterministic binary encoding of document state via `Y.encodeStateAsUpdate`, (d) mutable `clientId` property on `Y.Doc` (Automerge does not allow changing `actorId` after document creation), (e) Lamport clock starting at 0 for fresh documents. The general principle — content-addressed replica identifiers for structural operations — is portable to any operation-based CRDT with item deduplication. The specific implementation assumes Yjs's YATA algorithm.

7 Related Work

Cambria^[9] is the closest prior art. Its “phantom change” mechanism is structurally similar to genesis. Genesis makes the mechanism permanent and persistent. However, Cambria provides a capability genesis does not: bidirectional lenses that *transform* data between schema versions. Genesis handles convergent initialization (additive evolution) but has no story for schema transformation (renames, type changes). The two approaches are complementary — if Cambria were revived with genesis as its initialization primitive, the phantom change fragility that limited the original implementation would be eliminated.

Weidner’s CRDT-Valued Map^[12] formalizes the lazy-materialization pattern: “every key/value pair is implicitly always present in the map, but values are only explicitly constructed in memory as needed, using a predefined factory method.” Weidner’s pattern already implies value-convergence (the factory produces CRDTs, and CRDTs converge). Genesis adds *identity*-convergence (same identifier across peers), positional priority (genesis items cannot be displaced), and undo invisibility — properties beyond the pattern’s formalization.

EVM CREATE2^[13] provides a useful analogy. A smart contract’s address is derived from `hash(deployer, salt, bytecode)`, enabling counterfactual interaction with an address before the contract exists. Genesis derives an entity’s identity from `hash(parent, field, key, shape)`, enabling independent peers to write to an entity before it is materialized. The correspondence is structural, not exact: CREATE2 produces immutable bytecode, while genesis entities are mutable CRDT objects whose content evolves after creation.

Automerger’s `objectId` already derives object identity from `{actorId, counter}`^[6,14], collapsing identity and creating-operation coordinates at the CRDT level — an insight we share with the entity-oriented document model^[2]. Genesis takes the further step of making the `actorId` itself content-addressed, so the identity is deterministic across peers rather than merely per-creator stable. The composition with the partitioning scheme^[1] is what makes this safe: genesis identifiers live in a dedicated range, so content-addressed `actorIds` cannot collide with random peer `actorIds`.

Yjs pre-encoded templates^[3] and **Automerger hard-coded byte arrays**^[6] are the production workarounds for deterministic initialization. Both require build-time knowledge of the initial state and cannot handle per-entity or schema-evolution scenarios.

Genesis generalizes the pattern: every structural position has its own deterministic “template” computed at runtime from the schema.

Semantic identifier partitioning^[1] — itself an adaptation of Yu et al.’s categorical-second-dimension technique^[15] to op-based CRDTs — provides the namespace that makes genesis items structurally distinct from user operations. Self-resolving entity identifiers (§5 of^[2]) provide the `d`-prefix encoding that makes genesis entities addressable and cross-referenceable.

DXOS ECHO Epochs^[16] address schema migration through periodic “new beginnings” that compact history. Unlike genesis, Epochs require coordination (only one of two concurrent epoch creations survives) and operate at document granularity, not per-entity.

Merkle-CRDTs^[17] use content-addressed hashing for operation deduplication — hashing operation bytes, not the semantic content of created entities. Genesis uses content-addressing at a different level: deriving entity *identity* from structural position, enabling convergent creation rather than deduplication of existing operations.

Yjs subdocuments. A reviewer familiar with Yjs internals may ask why genesis doesn’t use subdocs (which give each sub-document its own `clientId` space). Subdocs are a runtime containment primitive for sync-scoping and lazy loading, not an identity-convergence primitive: two peers independently creating the “same” subdoc still produce distinct subdoc instances with distinct `clientIds`. Subdocs solve a different problem.

8 Conclusion

The initialization problem in decentralized CRDTs is a structural deficiency that every production system works around with ad hoc patterns, each with known failure modes. The underlying issue is that CRDTs treat structural initialization and user content creation as the same operation, when they have different convergence requirements.

Deterministic genesis gives structural operations their own identity mechanism: content-addressed replica identifiers that produce byte-identical CRDT items regardless of which peer executes the initialization. The question of whether a given structural entity exists ceases to be a coordination question and becomes a pure computation over the schema and the structural position.

The costs are real: factory purity is partially enforced, the technique is Yjs-specific in its current form, state vectors inflate with genesis path count, and non-additive

schema changes still require coordination. The benefits are that initialization cannot be undone, cannot race, cannot duplicate, and converges without communication — properties that the existing workarounds each provide partially, and that genesis provides together.

References

- [1] Plexus Project. Adapting categorical CRDT lifecycle to op-based CRDTs via replica-identifier partitioning. Preprint, 2026.
- [2] Plexus Project. Entity-oriented CRDT documents: Architecture, identity, and structural liveness. Preprint, 2026.
- [3] Kevin Jahns. Initial offline value of a shared document. Yjs Community Forum, discuss.yjs.dev, 2021.
- [4] Braden. What is the correct way to apply document migrations? Yjs Community Forum, discuss.yjs.dev, 2024.
- [5] Stephen Dolan. The only undoable CRDTs are counters. *arXiv:2006.10494*, 2020.
- [6] Automerge. Modeling data. <https://automerge.org/docs/cookbook/modeling-data/>, 2024.
- [7] Liveblocks. Setting an initial or default value in BlockNote. <https://liveblocks.io/docs/guides/>, 2024.
- [8] Evan Wallace. How figma’s multiplayer technology works. <https://figma.com/blog>, 2019.
- [9] Geoffrey Litt, Peter van Hardenberg, and Orion Henry. Cambria: Schema evolution in distributed systems with edit lenses. In *Proc. PaPoC (ACM)*, 2021.
- [10] Plexus Project. Liminal state: Deferred-persistence lifecycles for operation-based CRDTs. Preprint, 2026.
- [11] Petru Nicolaescu, Kevin Jahns, Michael Derntl, and Ralf Klamma. Near real-time peer-to-peer shared editing on extensible data types. In *Proc. ACM GROUP*, 2016.
- [12] Matthew Weidner. Designing data structures for collaborative apps. <https://mattweidner.com>, 2022.
- [13] Vitalik Buterin. EIP-1014: Skinny CREATE2. Ethereum Improvement Proposals, 2018.
- [14] Martin Kleppmann and Alastair R. Beresford. A conflict-free replicated JSON datatype. *IEEE TPDS*, 2017.
- [15] Weihai Yu, Victorien Elvinger, and Claudia-Lavinia Ignat. A generic undo support for state-based CRDTs. In *Proc. OPODIS*, 2019.
- [16] DXOS. Decentralized schema changes and data migrations. <https://blog.dxos.org>, 2024.
- [17] Héctor Sanjuán, Samuli Pöyhkäri, and Pedro Teixeira. Merkle-CRDTs: Merkle-DAGs meet CRDTs. Protocol Labs, 2020.